

## TP N°3: Initiation à l'ESP32 sur Wokwi

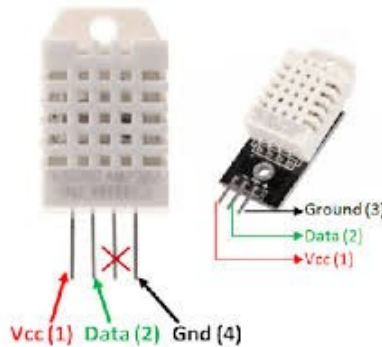
### I. Objectifs pédagogiques

Découvrir les fonctionnalités principales de la carte ESP32 à travers une série d'exercices pratiques de complexité croissante sur le simulateur **Wokwi**.

### II. Exercice 1 : Capteur DHT22 (température et humidité)

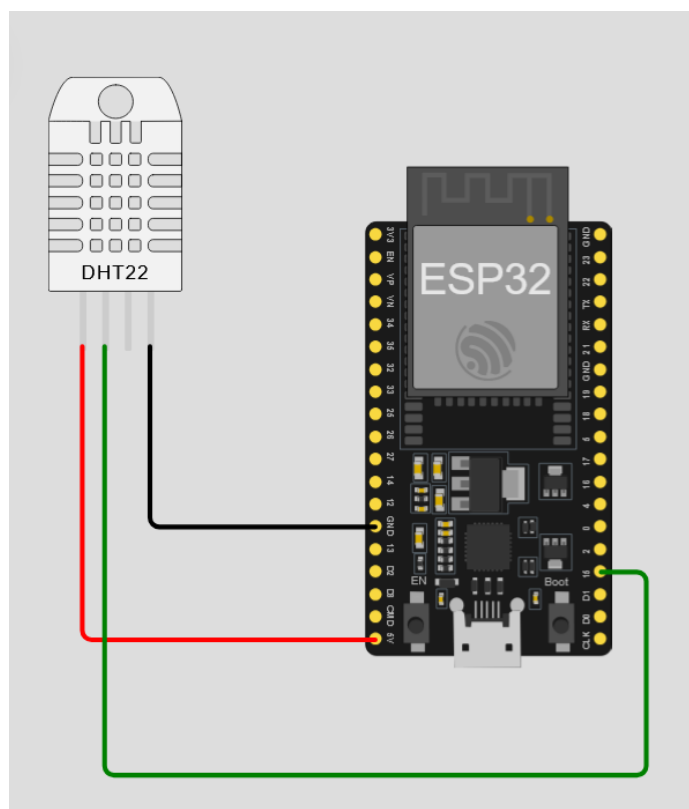
Composants :

- Capteur **DHT22**
- Broche de données sur GPIO 15



DHT22 broches

- Câblage



- Code

```
1  #include "DHT.h"
2
3  #define DHTPIN 15
4  #define DHTTYPE DHT22
5
6  DHT dht(DHTPIN, DHTTYPE);
7
8  void setup() {
9      Serial.begin(115200);
10     dht.begin();
11 }
12
13 void loop() {
14     float h = dht.readHumidity();
15     float t = dht.readTemperature();
16
17     Serial.print("Humidité: ");
18     Serial.print(h);
19     Serial.print(" %  Température: ");
20     Serial.print(t);
21     Serial.println(" °C");
22     delay(2000);
23 }
```

**Simulation :**

Simulation

00:05.429 83%

load:0x3fff0030,len:1156  
load:0x40078000,len:11456  
ho 0 tail 12 room 4  
load:0x40080400,len:2972  
entry 0x400805dc  
Humidité: 40.00 % Température: 24.00 °C  
Humidité: 40.00 % Température: 24.00 °C

1. Tester le code et afficher le moniteur série pour visualiser la sortie série de l'ESP.
2. Modifier le code selon les consignes suivantes :
  - a. Ajouter un code qui affiche un message d'alerte si la température dépasse 30°C ;  
Sinon, afficher un message de confort 'Température Normale'
  - b. Stocker les 5 dernières mesures de température dans un tableau.
  - c. Calculer la moyenne pour lisser les valeurs avant l'affichage.
  - d. Créer un diagnostic de confort basé sur les deux capteurs :  
Si  $t > 30$  et  $h < 40 \rightarrow$  "Air chaud et sec"  
Si  $t < 20$  et  $h > 60 \rightarrow$  "Air froid et humide"  
Sinon  $\rightarrow$  "Conditions normales"
  - e. Lire le capteur toutes les 10 secondes au lieu de 2.  
Allumer une LED intégrée (GPIO 2) seulement pendant la lecture.

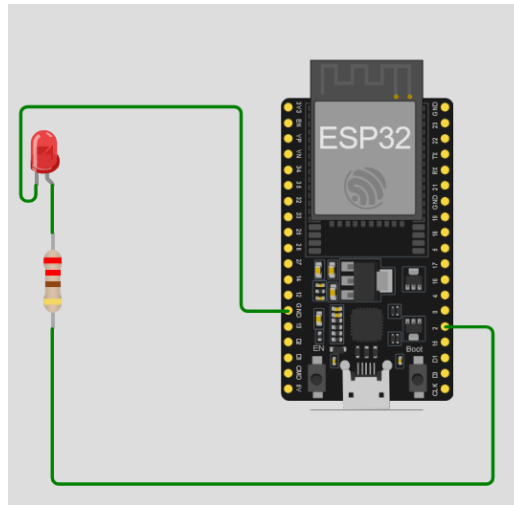
### III. Exercice 2: Contrôle à distance via le moniteur série

**Objectif :** Allumer ou éteindre la LED selon une commande envoyée via le moniteur série.

**Composants :**

- 1 LED (GPIO 2)
- Résistance ( 220 ohms)

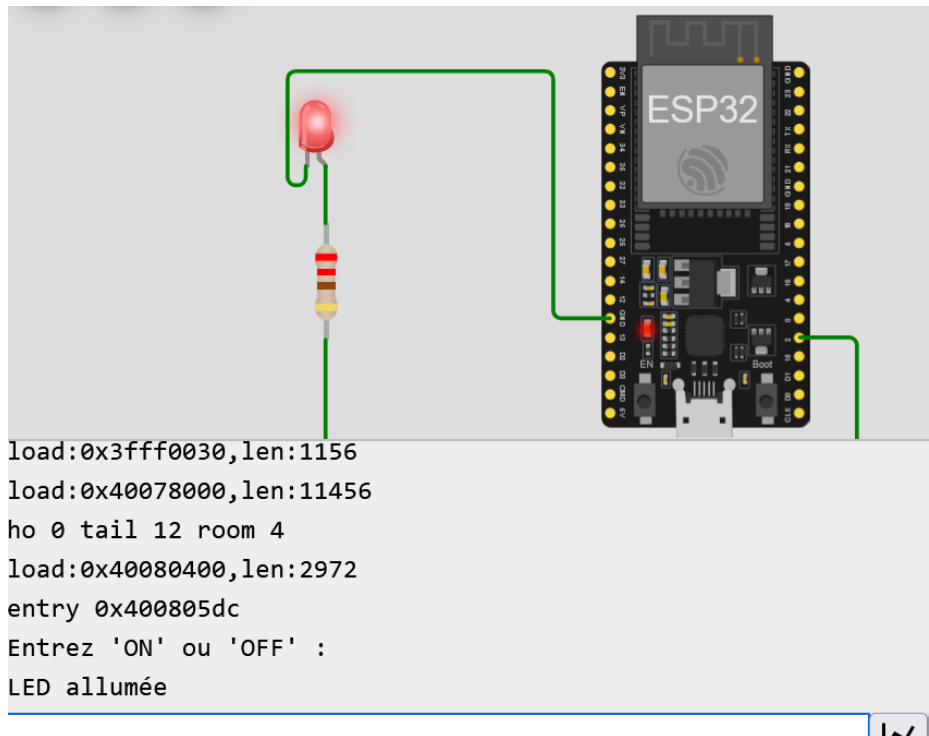
1. Réaliser le schéma suivant :



2. Tapez le code suivant :

```
1  #define LED_PIN 2
2  String command;
3
4  void setup() {
5      pinMode(LED_PIN, OUTPUT);
6      Serial.begin(115200);
7      Serial.println("Entrez 'ON' ou 'OFF' :");
8  }
9
10 void loop() {
11     if (Serial.available()) {
12         command = Serial.readStringUntil('\n');
13         command.trim();
14
15         if (command == "ON") {
16             digitalWrite(LED_PIN, HIGH);
17             Serial.println("LED allumée");
18         } else if (command == "OFF") {
19             digitalWrite(LED_PIN, LOW);
20             Serial.println("LED éteinte");
21         } else {
22             Serial.println("Commande inconnue !");
23         }
24     }
25 }
```

3. Faire la simulation avec le moniteur série : Tapez ON/OFF pour allumer ou éteindre la LED.



4. Créer une commande supplémentaire "BLINK" qui fait clignoter la LED 5 fois.
5. Afficher au démarrage la liste des commandes disponibles :

Commandes disponibles :

- ON : allumer la LED
- OFF : éteindre la LED
- BLINK : clignoter 5 fois
- STATUS : afficher l'état de la LED

6. Gérer la commande "STATUS" pour afficher si la LED est actuellement allumée ou éteinte.

Aide mémoire :

| Fonction                         | Rôle                                                             |
|----------------------------------|------------------------------------------------------------------|
| Serial.begin(115200);            | Initialise la communication série                                |
| Serial.available()               | Vérifie si une commande est arrivée                              |
| Serial.readStringUntil('\n');    | Lit la ligne saisie par l'utilisateur                            |
| String.trim()                    | Supprime les espaces et retours à la ligne                       |
| String.equalsIgnoreCase("texte") | Compare deux chaînes sans tenir compte des majuscules/minuscules |
| pinMode(pin, OUTPUT)             | Définit une broche en sortie                                     |
| digitalWrite(pin, HIGH/LOW)      | Allume ou éteint la LED                                          |
| for (int i=0; i<5; i++) {...}    | Répéter une action (ex : clignotement)                           |
| delay(ms)                        | Attendre un certain temps (en millisecondes)                     |